INTRO

Introduction

Introduction aux TPs

Application fil rouge : Smart Campus Dashboard

Au fil des 9 TPs, vous allez construire un **tableau de bord de gestion du campus** qui permet de :

- Visualiser et filtrer la liste des salles (capacité, occupation, équipements)
- Suivre les projets étudiants (PFE, hackathons, innovation)
- Consulter les capteurs IoT des bâtiments (température, CO_2, présence)
- Voir la consommation énergétique par bâtiment

Chaque TP ajoute une couche de fonctionnalité supplémentaire :

```
TP1: Setup environnement (Vite + Vue 3 + TypeScript)
TP2: Vue statique → réactive (template syntax)
TP3: Composants Options API
TP4: Refactor en Composition API + composables
TP5: Multi-pages avec Vue Router
TP6: Données depuis une API REST mock
TP7: État centralisé avec Pinia
TP8: Composable typé + directive custom
TP9: Tests Vitest
```

Environnement requis

- Node.js 22 LTS+ (`node --version`)
- npm 10+ (`npm --version`)

- VS Code + extension **Vue (Volar)**
- Vue DevTools (extension navigateur)

Données mock

À partir du TP6, vous lancerez un serveur mock JSON local :

```
cd Exercises/mock-server
npm install
npm start              # http://localhost:4001
```

Endpoints disponibles :

- `GET /rooms`
- `GET /rooms/:id`
- `POST /rooms/:id/favorite`
- `GET /projects`
- `GET /sensors`
- `GET /energy`

Conventions de code

- TypeScript dès que possible (TP1 inclus)
- `<script setup lang="ts">` pour tous les composants à partir du TP4
- Style : Bulma CSS + Font Awesome icons (intégrés dans le squelette)
- ESLint + Prettier configurés dans chaque lab

TP1

Setup

TP1 — Setup environnement (30 min)

Modules associés : M02 — Instance, M03 — Tooling

Objectif

Scaffolder un projet Vue 3 + Vite + TypeScript depuis zéro avec `create-vue`, le démarrer en mode dev et personnaliser la page d'accueil pour le **Smart Campus Dashboard**.

Pré-requis

- Node.js 22 LTS installé
- VS Code + extension Vue (Volar)

Étapes

1. Vérifier l'environnement

```
node --version    # doit être ≥ 22.0
npm --version     # doit être ≥ 10.0
```

2. Scaffolder le projet

Depuis le dossier `Exercises/workspaces/lab1` (ou tout autre dossier de travail) :

```
npm create vue@latest campus-dashboard
```

Répondez aux questions :

- ✅ TypeScript ? **Yes**
- ❌ JSX ? **No**
- ✅ Vue Router ? **Yes** (on l'utilisera au TP5)
- ✅ Pinia ? **Yes** (utilisé au TP7)
- ✅ Vitest ? **Yes** (utilisé au TP9)
- ❌ End-to-End testing ? **No**
- ✅ ESLint ? **Yes**
- ✅ Prettier ? **Yes**

```
cd campus-dashboard
npm install
npm run dev
```

Ouvrir http://localhost:5173 — vous devriez voir la page d'accueil par défaut de Vue.

3. Récupérer la feuille de style commune

Copiez `campus.css` (fourni dans `Exercises/corrections/lab1/src/assets/campus.css`) vers votre projet :

```
mkdir -p src/assets
cp ../../../corrections/lab1/src/assets/campus.css src/assets/campus.css
```

Cette feuille de style définit le **design system** du Smart Campus (palette de couleurs, cartes, badges, notifications, etc.). Vous n'aurez plus à écrire de CSS dans les TPs suivants — concentrez-vous sur le code Vue.

4. Importer `campus.css` et personnaliser l'app

Dans `src/main.ts`, ajoutez l'import du CSS :

```
import { createApp } from 'vue';
import App from './App.vue';
import './assets/campus.css';        // ← style commun du Campus

createApp(App).mount('#app');
```

Dans `index.html`, forcez le thème clair et chargez la police Inter :

```
<!doctype html>
<html lang="fr" data-theme="light">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0"
    <title>Smart Campus Dashboard</title>
    <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;50
  </head>
  <body>
    <div id="app"></div>
    <script type="module" src="/src/main.ts"></script>
  </body>
</html>
```

> Notez le `data-theme="light"` : il évite que la palette s'adapte au mode sombre du système d'exploitation pendant la formation.

5. Personnaliser `App.vue`

Remplacez le contenu de `src/App.vue` par :

```
<script setup lang="ts">
import { ref } from 'vue';

const campusName = ref('Polytech Paris-Sud');
const today = new Date().toLocaleDateString('fr-FR', {
  weekday: 'long', year: 'numeric', month: 'long', day: 'numeric'
});
const counter = ref(0);
</script>

<template>
  <header class="campus-navbar">
    <span class="brand">🏫 Smart Campus Dashboard</span>
  </header>

  <main class="page">
    <div class="page-header">
      <h1>Bienvenue sur le campus de {{ campusName }}</h1>
      <p class="subtitle">{{ today }}</p>
    </div>

    <div class="detail-box" style="text-align: center;">
      <h2>Démo réactivité</h2>
      <p style="font-size: 4rem; font-weight: 700; color: var(--vue-green);">{
      <div style="display: flex; gap: 8px; justify-content: center;">
        <button class="btn" @click="counter--">− 1</button>
        <button class="btn" @click="counter = 0">Reset</button>
        <button class="btn btn-primary" @click="counter++">+ 1</button>
      </div>
    </div>
  </main>
</template>
```

Supprimez les fichiers inutiles : `src/components/HelloWorld.vue`, `src/components/TheWelcome.vue`, `src/components/WelcomeItem.vue`,

`src/components/icons/`, et le contenu de `src/assets/main.css` / `src/assets/base.css` (ou supprimer les imports correspondants dans `main.ts`).

6. Vérifier le build et le type-check

```
npm run build
npm run preview
npm run type-check
```

Tout doit passer sans erreur.

Livrable

- Un projet `campus-dashboard/` qui démarre avec `npm run dev`
- `src/assets/campus.css` présent et importé dans `main.ts`
- L'affichage de la page d'accueil personnalisée avec le compteur réactif
- `npm run build` et `npm run type-check` passent

Bonus

- Ouvrez les **Vue DevTools** dans le navigateur et inspectez le composant `App`.
- Modifiez la valeur initiale de `counter` et observez le HMR (Hot Module Replacement) qui met à jour la page sans la recharger.

TP suivant : TP2 — Template syntax

TP2

Template syntax

TP2 — Template syntax (1 h)

Module associé : M04 — Template Syntax

Objectif

Transformer une page statique en application réactive : afficher dynamiquement une liste de salles du campus, gérer favoris, filtrage et statut d'occupation.

Pré-requis

- TP1 terminé (projet `campus-dashboard` en place)

Étapes

1. Préparer les données

Créer le fichier `src/data/rooms.ts` :

```
export interface Room {
  id: number;
  name: string;
  building: string;
  capacity: number;
  occupancy: number;
  equipment: string[];
  isFavorite: boolean;
}

export const rooms: Room[] = [
  { id: 1, name: 'Amphi Galois',    building: '510', capacity: 80, occupan
  { id: 2, name: 'Salle 101',       building: '620', capacity: 30, occupan
  { id: 3, name: 'Amphi Hadamard',  building: '510', capacity: 120, occupa
  { id: 4, name: 'Salle TP Info A', building: '650', capacity: 24, occupan
  { id: 5, name: 'Salle Conférence',building: '510', capacity: 50, occupan
];
```

2. Afficher la liste avec v-for

Dans `src/App.vue` :

```
<script setup lang="ts">
import { ref } from 'vue';
import { rooms as initialRooms } from './data/rooms';

const rooms = ref(initialRooms);
</script>

<template>
  <header>
    <h1>🏫 Smart Campus Dashboard</h1>
  </header>

  <main>
    <h2>Salles du campus</h2>
    <ul class="card-grid">
      <li v-for="room in rooms" :key="room.id" class="room-card">
        <h3>{{ room.name }}</h3>
        <p>Bâtiment {{ room.building }} · {{ room.capacity }} places</p>
      </li>
    </ul>
  </main>
</template>
```

3. Filtrer les salles occupées (v-if)

Ajouter un statut visuel selon l'occupation :

```
<li v-for="room in rooms" :key="room.id" class="room-card">
  <h3>{{ room.name }}</h3>
  <p>Bâtiment {{ room.building }} · {{ room.capacity }} places</p>
  <p v-if="room.occupancy === 0" class="status-badge is-free">✅ Libre</p>
  <p v-else-if="room.occupancy < room.capacity * 0.8" class="status-badge is-pa
    🟡 Partiellement occupée ({{ room.occupancy }}/{{ room.capacity }})
  </p>
  <p v-else class="status-badge is-full">🔴 Pleine ({{ room.occupancy }}/{{ roo
</li>
```

4. Classes dynamiques (:class)

Coloriser la carte selon le statut :

```
<li
  v-for="room in rooms"
  :key="room.id"
  :class="['room-card', occupancyClass(room)]"
>
  ...
</li>
```

Ajouter dans le script :

```
function occupancyClass(room: Room) {
  if (room.occupancy === 0) return 'is-free';
  if (room.occupancy < room.capacity * 0.8) return 'is-partial';
  return 'is-full';
}
```

5. Bouton favoris (@click)

```
<li ...>
  ...
  <button class="fav-btn" @click="toggleFavorite(room.id)">
    {{ room.isFavorite ? '★' : '☆' }}
  </button>
</li>
```

```
function toggleFavorite(id: number) {
  const room = rooms.value.find(r => r.id === id);
  if (room) room.isFavorite = !room.isFavorite;
}
```

6. Champ de recherche (v-model)

Filtrer les salles par nom :

```
<div class="search-bar">
  <input v-model="searchTerm" placeholder="Rechercher une salle..." />
  <button v-if="searchTerm" class="clear-btn" @click="searchTerm = ''">×</butto
</div>
```

```
import { computed } from 'vue';

const searchTerm = ref('');
const filteredRooms = computed(() =>
  rooms.value.filter(r =>
    r.name.toLowerCase().includes(searchTerm.value.toLowerCase())
  )
);
```

Remplacer `v-for="room in rooms"` par `v-for="room in filteredRooms"`.

7. Toggle « afficher seulement les favoris » (v-show)

```
<label>
  <input type="checkbox" v-model="onlyFavorites" />
  Mes favoris uniquement
</label>
```

Mettre à jour le computed :

```
const onlyFavorites = ref(false);
const filteredRooms = computed(() =>
  rooms.value
    .filter(r => r.name.toLowerCase().includes(searchTerm.value.toLowerCas
    .filter(r => !onlyFavorites.value || r.isFavorite)
);
```

Livrable

- L'application affiche les 5 salles
- Bouton favori cliquable, fonctionne
- Champ de recherche filtre en temps réel
- Toggle « favoris uniquement » fonctionnel
- Couleurs dynamiques selon l'occupation

Bonus

- Trier par capacité (asc / desc) avec un `<select>` v-model
- Ajouter un compteur « N résultats affichés »
- Utiliser `v-for` imbriqué pour afficher la liste des équipements

---

TP suivant : TP3 — Options API

TP3

Options API

TP3 — Options API (1 h)

Modules associés : M05 — Options API, M07 — Communication composants

Objectif

Extraire la carte de salle dans un composant `RoomCard.vue` écrit en **Options API**. Pratiquer props, computed, methods, emit.

Pré-requis

- TP2 terminé

Étapes

1. Créer RoomCard.vue (Options API)

`src/components/RoomCard.vue` :

```
<script lang="ts">
import { defineComponent, type PropType } from 'vue';
import type { Room } from '../data/rooms';

export default defineComponent({
  name: 'RoomCard',
  props: {
    room: {
      type: Object as PropType<Room>,
      required: true
    }
  },
  emits: ['toggle-favorite'],
  computed: {
    occupancyRate(): number {
      return (this.room.occupancy / this.room.capacity) * 100;
    },
    occupancyClass(): string {
      if (this.room.occupancy === 0) return 'is-free';
      if (this.occupancyRate < 80) return 'is-partial';
      return 'is-full';
    },
    statusLabel(): string {
      if (this.room.occupancy === 0) return '✅ Libre';
      if (this.occupancyRate < 80) return `🟡 ${this.room.occupancy}/${this.roo
      return `🔴 ${this.room.occupancy}/${this.room.capacity}`;
    }
  },
  methods: {
    onFavoriteClick() {
      this.$emit('toggle-favorite', this.room.id);
    }
  }
});
</script>

<template>
  <article :class="['room-card', occupancyClass]">
    <header>
      <h3>{{ room.name }}</h3>
      <button class="fav-btn" @click="onFavoriteClick">
```

```
        {{ room.isFavorite ? '★' : '☆' }}
      </button>
    </header>
    <p>Bâtiment {{ room.building }} · {{ room.capacity }} places</p>
    <p class="status-badge">{{ statusLabel }}</p>
    <ul class="equipment-tags">
      <li v-for="eq in room.equipment" :key="eq">{{ eq }}</li>
    </ul>
  </article>
</template>
```

2. Utiliser RoomCard depuis App.vue

```
<script setup lang="ts">
import { ref, computed } from 'vue';
import RoomCard from './components/RoomCard.vue';
import { rooms as initialRooms } from './data/rooms';

const rooms = ref(initialRooms);
const searchTerm = ref('');

const filteredRooms = computed(() =>
  rooms.value.filter(r => r.name.toLowerCase().includes(searchTerm.value.toLowe
);

function onToggleFavorite(id: number) {
  const room = rooms.value.find(r => r.id === id);
  if (room) room.isFavorite = !room.isFavorite;
}
</script>

<template>
  <header>
    <h1>🏫 Smart Campus Dashboard</h1>
    <input v-model="searchTerm" placeholder="Rechercher une salle..." />
  </header>

  <main class="card-grid">
    <RoomCard
      v-for="room in filteredRooms"
      :key="room.id"
      :room="room"
      @toggle-favorite="onToggleFavorite"
    />
  </main>
</template>
```

3. Ajouter un watcher pour logger les changements

Dans `RoomCard.vue`, ajouter une option `watch` :

```
watch: {
  'room.isFavorite'(newVal: boolean) {
    console.log(`[${this.room.name}] favori = ${newVal}`);
  }
}
```

4. Lifecycle : log au montage

```
mounted() {
  console.log(`RoomCard monté pour ${this.room.name}`);
},
beforeUnmount() {
  console.log(`RoomCard démonté pour ${this.room.name}`);
}
```

Bonus

- Créer un composant `<SearchBar>` en Options API qui émet `update:searchTerm` (préfiguration du TP4 avec v-model custom).
- Ajouter une prop `compact: Boolean` pour un mode d'affichage condensé.
- Validateur de prop : `room.capacity > 0`.

Livrable

- `RoomCard.vue` autonome, configurable via prop `room`, émet `toggle-favorite`
- L'app fonctionne comme au TP2 mais découpée en composants
- Console montre les logs de cycle de vie

TP suivant : TP4 — Composition API

TP4

Composition API

TP4 — Composition API (40 min)

Modules associés : M06 — Composition API, M07 — Composants

Objectif

Refactor `RoomCard.vue` en `<script setup>` (Composition API), extraire la logique de recherche dans un composable `useSearch`.

Pré-requis

- TP3 terminé

Étapes

1. Refactor RoomCard en `<script setup>`

`src/components/RoomCard.vue` (nouvelle version) :

```
<script setup lang="ts">
import { computed, onMounted, onBeforeUnmount, watch } from 'vue';
import type { Room } from '../data/rooms';

const props = defineProps<{ room: Room }>();
const emit = defineEmits<{
  'toggle-favorite': [id: number];
}>();

const occupancyRate = computed(() => (props.room.occupancy / props.room.capaci

const occupancyClass = computed(() => {
  if (props.room.occupancy === 0) return 'is-free';
  if (occupancyRate.value < 80) return 'is-partial';
  return 'is-full';
});

const statusLabel = computed(() => {
  if (props.room.occupancy === 0) return '✅ Libre';
  return `${props.room.occupancy}/${props.room.capacity}`;
});

function onFavoriteClick() {
  emit('toggle-favorite', props.room.id);
}

watch(() => props.room.isFavorite, (newVal) => {
  console.log(`[${props.room.name}] favori = ${newVal}`);
});

onMounted(() => console.log(`RoomCard monté : ${props.room.name}`));
onBeforeUnmount(() => console.log(`RoomCard démonté : ${props.room.name}`));
</script>

<template>
  <article :class="['room-card', occupancyClass]">
    <header>
      <h3>{{ room.name }}</h3>
      <button class="fav-btn" @click="onFavoriteClick">
        {{ room.isFavorite ? '★' : '☆' }}
      </button>
```

```
    </header>
    <p>Bâtiment {{ room.building }} · {{ room.capacity }} places</p>
    <p class="status-badge">{{ statusLabel }}</p>
  </article>
</template>
```

2. Créer le composable useSearch

`src/composables/useSearch.ts` :

```
import { ref, computed, type Ref } from 'vue';

export function useSearch<T>(items: Ref<T[]>, key: keyof T) {
  const term = ref('');

  const filtered = computed(() =>
    items.value.filter(item =>
      String(item[key]).toLowerCase().includes(term.value.toLowerCase())
    )
  );

  function clear() {
    term.value = '';
  }

  return { term, filtered, clear };
}
```

3. Utiliser useSearch dans App.vue

```
<script setup lang="ts">
import { ref } from 'vue';
import RoomCard from './components/RoomCard.vue';
import { useSearch } from './composables/useSearch';
import { rooms as initialRooms } from './data/rooms';

const rooms = ref(initialRooms);
const { term: searchTerm, filtered: filteredRooms, clear } = useSearch(rooms,

function onToggleFavorite(id: number) {
  const room = rooms.value.find(r => r.id === id);
  if (room) room.isFavorite = !room.isFavorite;
}
</script>

<template>
  <header>
    <h1>🏫 Smart Campus Dashboard</h1>
    <div class="search-bar">
      <input v-model="searchTerm" placeholder="Rechercher une salle..." />
      <button @click="clear" v-if="searchTerm">x</button>
    </div>
  </header>

  <main class="card-grid">
    <RoomCard
      v-for="room in filteredRooms"
      :key="room.id"
      :room="room"
      @toggle-favorite="onToggleFavorite"
    />
  </main>
</template>
```

4. Créer SearchInput avec defineModel

`src/components/SearchInput.vue` :

```
<script setup lang="ts">
const model = defineModel<string>();

defineProps<{ placeholder?: string }>();
</script>

<template>
  <div class="search-bar">
    <input v-model="model" :placeholder="placeholder" />
    <button @click="model = ''" v-if="model">×</button>
  </div>
</template>
```

Dans `App.vue` :

```
<SearchInput v-model="searchTerm" placeholder="Rechercher..." />
```

Bonus

- Étendre `useSearch` pour accepter plusieurs clés (`['name', 'building']`).
- Ajouter un `useFavorites` composable qui filtre les favoris.
- Combiner `useSearch + useFavorites` dans `App.vue` (composition !).

Livrable

- `RoomCard.vue` en `<script setup>`
- Composable `useSearch.ts` réutilisable et typé
- Composant `SearchInput.vue` avec `defineModel`
- App identique fonctionnellement mais code plus modulaire

TP suivant : TP5 — Vue Router

TP5

Vue Router

TP5 — Vue Router (45 min)

Module associé : M08 — Vue Router 4

Objectif

Transformer la SPA mono-page en application multi-pages : pages d'accueil, salles, détails d'une salle, projets étudiants.

Pré-requis

- TP4 terminé
- Vue Router déjà installé (choix du wizard `create-vue` au TP1)

Étapes

1. Configurer le routeur

`src/router/index.ts` (existe peut-être déjà — adapter) :

```
import { createRouter, createWebHistory } from 'vue-router';
import HomeView from '../views/HomeView.vue';

const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    { path: '/',          name: 'home',        component: HomeView },
    { path: '/rooms',     name: 'rooms',       component: () => import('..
    { path: '/rooms/:id', name: 'room-detail', component: () => import('..
    { path: '/projects',  name: 'projects',    component: () => import('..
    { path: '/:pathMatch(.*)*', name: 'not-found', component: () => import
  ]
});

export default router;
```

2. App.vue : navigation + RouterView

```
<script setup lang="ts">
import { RouterLink, RouterView } from 'vue-router';
</script>

<template>
  <header class="campus-navbar">
    <h1>🏫 Smart Campus</h1>
    <nav>
      <RouterLink to="/">Accueil</RouterLink>
      <RouterLink to="/rooms">Salles</RouterLink>
      <RouterLink to="/projects">Projets</RouterLink>
    </nav>
  </header>

  <main>
    <RouterView />
  </main>
</template>

<style>
.router-link-active { color: #42b883; font-weight: bold; }
</style>
```

3. Créer les vues

`src/views/HomeView.vue` — Tableau de bord d'accueil :

```
<script setup lang="ts">
import { rooms } from '../data/rooms';

const stats = {
  totalRooms: rooms.length,
  totalCapacity: rooms.reduce((sum, r) => sum + r.capacity, 0),
  occupancy: rooms.reduce((sum, r) => sum + r.occupancy, 0)
};
</script>

<template>
  <section>
    <h2>Bienvenue sur le Smart Campus Dashboard</h2>
    <div class="stats-grid">
      <div class="stat-card">
        <h3>{{ stats.totalRooms }}</h3>
        <p>Salles</p>
      </div>
      <div class="stat-card">
        <h3>{{ stats.totalCapacity }}</h3>
        <p>Places totales</p>
      </div>
      <div class="stat-card">
        <h3>{{ stats.occupancy }}</h3>
        <p>Personnes présentes</p>
      </div>
    </div>
  </section>
</template>
```

`src/views/RoomsView.vue` — Liste des salles (déplacer le code de App.vue précédent) :

```
<script setup lang="ts">
import { ref } from 'vue';
import RoomCard from '../components/RoomCard.vue';
import { useSearch } from '../composables/useSearch';
import { rooms as initialRooms } from '../data/rooms';

const rooms = ref(initialRooms);
const { term, filtered } = useSearch(rooms, 'name');

function onToggle(id: number) {
  const r = rooms.value.find(x => x.id === id);
  if (r) r.isFavorite = !r.isFavorite;
}
</script>

<template>
  <h2>Salles du campus</h2>
  <input v-model="term" placeholder="Rechercher..." />
  <ul class="card-grid">
    <li v-for="room in filtered" :key="room.id">
      <RouterLink :to="{ name: 'room-detail', params: { id: room.id } }">
        <RoomCard :room="room" @toggle-favorite="onToggle" />
      </RouterLink>
    </li>
  </ul>
</template>
```

`src/views/RoomDetailView.vue` — Détail d'une salle :

```
<script setup lang="ts">
import { computed } from 'vue';
import { useRouter } from 'vue-router';
import { rooms } from '../data/rooms';

const props = defineProps<{ id: string }>();
const router = useRouter();

const room = computed(() => rooms.find(r => r.id === Number(props.id)));
</script>

<template>
  <button @click="router.back()">← Retour</button>
  <div v-if="room">
    <h2>{{ room.name }}</h2>
    <p>Bâtiment {{ room.building }}</p>
    <p>Capacité : {{ room.capacity }} places</p>
    <p>Occupation actuelle : {{ room.occupancy }}</p>
    <h3>Équipements</h3>
    <ul>
      <li v-for="eq in room.equipment" :key="eq">{{ eq }}</li>
    </ul>
  </div>
  <p v-else class="notice danger">Salle introuvable</p>
</template>
```

`src/views/ProjectsView.vue` — Liste des projets :

```
<script setup lang="ts">
const projects = [
  { id: 1, title: 'Smart Bin Sorter', status: 'En cours', team: ['Alice', 'Bob
  { id: 2, title: 'Drone Inspector', status: 'Terminé', team: ['Charlie'] },
  { id: 3, title: 'Energy Optimizer', status: 'En cours', team: ['Dora', 'Eve'
];
</script>

<template>
  <h2>Projets étudiants</h2>
  <ul>
    <li v-for="p in projects" :key="p.id">
      <strong>{{ p.title }}</strong> — {{ p.status }} ({{ p.team.join(', ') }}
    </li>
  </ul>
</template>
```

`src/views/NotFoundView.vue` :

```
<template>
  <h2>404 — Page introuvable</h2>
  <RouterLink to="/">Retour à l'accueil</RouterLink>
</template>
```

4. Ajouter un guard de navigation

Dans `router/index.ts`, simuler une protection pour `/projects` :

```
router.beforeEach((to) => {
  if (to.name === 'projects' && !localStorage.getItem('user')) {
    alert('Veuillez vous connecter pour voir les projets');
    return { name: 'home' };
  }
});
```

Pour tester, ouvrir la console et faire `localStorage.setItem('user', 'me')`.

Bonus

- Ajouter une route `/rooms/:id/sensors` nested pour les capteurs.
- Animer les transitions de page avec `<RouterView v-slot>` + `<transition>`.
- Persister un `lastVisitedRoute` dans `localStorage`.

Livrable

- 4 pages navigables : Accueil, Salles, Détail d'une salle, Projets
- Navigation programmatique (back button) fonctionnelle
- Lazy loading actif (vérifier dans l'onglet Réseau)
- Page 404 sur URL inexistante

TP suivant : TP6 — Data fetching

TP6

Data fetching

TP6 — Data fetching (40 min)

Module associé : M09 — Data Fetching

Objectif

Remplacer les données statiques par des appels à une API REST mock. Gérer les états *loading / error / success*.

Pré-requis

- TP5 terminé
- Mock server lancé : `cd Exercises/mock-server && npm start` (port 4001)

Étapes

1. Vérifier le mock server

```
curl http://localhost:4001/rooms
# Doit retourner un tableau JSON
```

2. Créer un client API

`src/api/client.ts` :

```
const BASE_URL = import.meta.env.VITE_API_URL || 'http://localhost:4001';

export async function apiGet<T>(path: string): Promise<T> {
  const res = await fetch(`${BASE_URL}${path}`);
  if (!res.ok) throw new Error(`HTTP ${res.status} — ${res.statusText}`);
  return res.json();
}

export async function apiPost<T>(path: string, body: unknown): Promise<T>
  const res = await fetch(`${BASE_URL}${path}`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(body)
  });
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  return res.json();
}
```

3. Refactor RoomsView avec fetch

`src/views/RoomsView.vue` :

```
<script setup lang="ts">
import { ref, onMounted } from 'vue';
import RoomCard from '../components/RoomCard.vue';
import { useSearch } from '../composables/useSearch';
import { apiGet } from '../api/client';
import type { Room } from '../data/rooms';

const rooms = ref<Room[]>([]);
const loading = ref(true);
const error = ref<string | null>(null);

const { term, filtered } = useSearch(rooms, 'name');

onMounted(async () => {
  try {
    rooms.value = await apiGet<Room[]>('/rooms');
  } catch (e) {
    error.value = (e as Error).message;
  } finally {
    loading.value = false;
  }
});

async function onToggle(id: number) {
  const r = rooms.value.find(x => x.id === id);
  if (r) r.isFavorite = !r.isFavorite;
  // Optionnel : persister via POST
  // await apiPost(`/rooms/${id}/favorite`, { isFavorite: r.isFavorite });
}
</script>

<template>
  <h2>Salles du campus</h2>
  <input v-model="term" placeholder="Rechercher..." />

  <p v-if="loading" class="notice info">⏳ Chargement des salles…</p>
  <p v-else-if="error" class="notice danger">❌ {{ error }}</p>
  <ul v-else class="card-grid">
    <li v-for="room in filtered" :key="room.id">
      <RoomCard :room="room" @toggle-favorite="onToggle" />
    </li>
```

```
    </ul>
  </template>
```

4. Charger une salle par ID

`src/views/RoomDetailView.vue` :

```
<script setup lang="ts">
import { ref, watch } from 'vue';
import { useRouter } from 'vue-router';
import { apiGet } from '../api/client';
import type { Room } from '../data/rooms';

const props = defineProps<{ id: string }>();
const router = useRouter();

const room = ref<Room | null>(null);
const loading = ref(true);
const error = ref<string | null>(null);

async function fetchRoom(id: string) {
  loading.value = true;
  error.value = null;
  try {
    room.value = await apiGet<Room>(`/rooms/${id}`);
  } catch (e) {
    error.value = (e as Error).message;
  } finally {
    loading.value = false;
  }
}

watch(() => props.id, fetchRoom, { immediate: true });
</script>

<template>
  <button @click="router.back()">← Retour</button>

  <p v-if="loading">⏳ Chargement…</p>
  <p v-else-if="error" class="notice danger">❌ {{ error }}</p>
  <article v-else-if="room">
    <h2>{{ room.name }}</h2>
    <p>Bâtiment {{ room.building }} · {{ room.capacity }} places</p>
    <p>Occupation : {{ room.occupancy }}</p>
  </article>
</template>
```

5. Tester la gestion d'erreur

Arrêter le mock server (Ctrl+C). Rafraîchir la page Salles : un message d'erreur doit s'afficher au lieu d'un écran blanc.

6. Variable d'environnement

`.env.local` à la racine du projet :

```
VITE_API_URL=http://localhost:4001
```

Adapter l'URL pour pointer vers un autre serveur si besoin.

Bonus

- Créer un composable `useApi<T>(endpoint)` qui encapsule `data / loading / error / refresh`.
- Ajouter un bouton « Rafraîchir » qui re-déclenche le fetch.
- Implémenter `AbortController` pour annuler les requêtes obsolètes.

Livrable

- L'application charge les données depuis l'API mock
- États loading / error correctement affichés
- Le détail d'une salle est récupéré par son ID

TP suivant : TP7 — Pinia

TP7

Pinia

TP7 — Pinia (30 min)

Module associé : M10 — Pinia

Objectif

Centraliser l'état des salles dans un store Pinia. Les composants ne tiennent plus l'état localement, mais consomment le store.

Pré-requis

- TP6 terminé
- Pinia déjà installé (choix du wizard `create-vue` au TP1)

Étapes

1. Créer le store campus

`src/stores/campus.ts` :

```
import { defineStore } from 'pinia';
import { ref, computed } from 'vue';
import { apiGet, apiPost } from '../api/client';
import type { Room } from '../data/rooms';

export const useCampusStore = defineStore('campus', () => {
  // State
  const rooms = ref<Room[]>([]);
  const loading = ref(false);
  const error = ref<string | null>(null);
  const searchTerm = ref('');

  // Getters
  const filteredRooms = computed(() =>
    rooms.value.filter(r =>
      r.name.toLowerCase().includes(searchTerm.value.toLowerCase())
    )
  );

  const favoriteRooms = computed(() => rooms.value.filter(r => r.isFavorit

  const totalCapacity = computed(() =>
    rooms.value.reduce((sum, r) => sum + r.capacity, 0)
  );

  // Actions
  async function fetchRooms() {
    loading.value = true;
    error.value = null;
    try {
      rooms.value = await apiGet<Room[]>('/rooms');
    } catch (e) {
      error.value = (e as Error).message;
    } finally {
      loading.value = false;
    }
  }

  async function toggleFavorite(id: number) {
    const room = rooms.value.find(r => r.id === id);
    if (!room) return;
    room.isFavorite = !room.isFavorite;
    // Optionnel : persister
```

```
    // await apiPost(`/rooms/${id}/favorite`, { isFavorite: room.isFavorit
  }

  function getRoomById(id: number) {
    return rooms.value.find(r => r.id === id);
  }

  return {
    rooms, loading, error, searchTerm,
    filteredRooms, favoriteRooms, totalCapacity,
    fetchRooms, toggleFavorite, getRoomById
  };
});
```

2. Initialiser le store dans App.vue

```
<script setup lang="ts">
import { onBeforeMount } from 'vue';
import { useCampusStore } from './stores/campus';

const campus = useCampusStore();
onBeforeMount(() => campus.fetchRooms());
</script>
```

3. Refactor RoomsView pour utiliser le store

```
<script setup lang="ts">
import { storeToRefs } from 'pinia';
import RoomCard from '../components/RoomCard.vue';
import { useCampusStore } from '../stores/campus';

const store = useCampusStore();
const { filteredRooms, searchTerm, loading, error } = storeToRefs(store);
const { toggleFavorite } = store;
</script>

<template>
  <h2>Salles du campus</h2>
  <input v-model="searchTerm" placeholder="Rechercher..." />

  <p v-if="loading">⏳ Chargement…</p>
  <p v-else-if="error" class="notice danger">❌ {{ error }}</p>
  <ul v-else class="card-grid">
    <li v-for="room in filteredRooms" :key="room.id">
      <RoomCard :room="room" @toggle-favorite="toggleFavorite" />
    </li>
  </ul>
</template>
```

4. Page Favoris (nouvelle vue)

`src/views/FavoritesView.vue` :

```
<script setup lang="ts">
import { storeToRefs } from 'pinia';
import RoomCard from '../components/RoomCard.vue';
import { useCampusStore } from '../stores/campus';

const store = useCampusStore();
const { favoriteRooms } = storeToRefs(store);
</script>

<template>
  <h2>⭐ Mes salles favorites</h2>
  <p v-if="favoriteRooms.length === 0">
    Aucune salle marquée favorite. Cliquez sur ☆ pour en ajouter.
  </p>
  <ul v-else class="card-grid">
    <li v-for="room in favoriteRooms" :key="room.id">
      <RoomCard :room="room" @toggle-favorite="store.toggleFavorite" />
    </li>
  </ul>
</template>
```

Ajouter la route dans `router/index.ts` :

```
{ path: '/favorites', name: 'favorites', component: () => import('../views
```

Lien dans la navbar de `App.vue` :

```
<RouterLink to="/favorites">Favoris</RouterLink>
```

5. Adapter RoomDetailView pour utiliser le store

```
<script setup lang="ts">
import { computed } from 'vue';
import { useCampusStore } from '../stores/campus';

const props = defineProps<{ id: string }>();
const store = useCampusStore();

const room = computed(() => store.getRoomById(Number(props.id)));
</script>

<template>
  <article v-if="room">
    <h2>{{ room.name }}</h2>
    <!-- ... -->
  </article>
  <p v-else>Salle introuvable</p>
</template>
```

6. Vérifier dans Vue DevTools

Onglet **Pinia** : on doit voir le store `campus` avec son state et ses getters.

Bonus

- Ajouter un store `auth` avec `user`, `login()`, `logout()`.
- Installer `pinia-plugin-persistedstate` pour persister les favoris.
- Créer une action `searchOnApi(term)` qui appelle un endpoint `/rooms?q=`.

Livrable

- Store `useCampusStore` opérationnel

- `RoomsView`, `RoomDetailView`, `FavoritesView` utilisent toutes le store
- DevTools Pinia montre le state et les mutations

TP suivant : TP8 — Réutilisabilité + TS

TP8

Reusability + TS

TP8 — Réutilisabilité + TypeScript (40 min)

Modules associés : M11 — Réutilisabilité, M12 — TypeScript

Objectif

Pratiquer l'écriture de composables typés et de directives custom. Créer un composable générique `useCampusApi<T>` et une directive `v-tooltip`.

Pré-requis

- TP7 terminé

Étapes

1. Composable générique useCampusApi

`src/composables/useCampusApi.ts` :

```typescript
import { ref, type Ref } from 'vue';
import { apiGet } from '../api/client';

export interface ApiState<T> {
  data: Ref<T | null>;
  loading: Ref<boolean>;
  error: Ref<string | null>;
  refresh: () => Promise<void>;
}

export function useCampusApi<T>(endpoint: string): ApiState<T> {
  const data = ref<T | null>(null) as Ref<T | null>;
  const loading = ref(false);
  const error = ref<string | null>(null);

  async function refresh() {
    loading.value = true;
    error.value = null;
    try {
      data.value = await apiGet<T>(endpoint);
    } catch (e) {
      error.value = (e as Error).message;
    } finally {
      loading.value = false;
    }
  }

  refresh();
  return { data, loading, error, refresh };
}
```

2. Utiliser useCampusApi pour les projets

Créer `src/types/project.ts` :

```
export interface Project {
  id: number;
  title: string;
  status: 'idea' | 'in-progress' | 'done';
  team: string[];
  tags: string[];
  description: string;
}
```

Refactor `src/views/ProjectsView.vue` :

```
<script setup lang="ts">
import { useCampusApi } from '../composables/useCampusApi';
import type { Project } from '../types/project';

const { data: projects, loading, error, refresh } = useCampusApi<Project[]>('/p
</script>

<template>
  <h2>Projets étudiants</h2>
  <button @click="refresh">🔄 Rafraîchir</button>

  <p v-if="loading">⏳ Chargement…</p>
  <p v-else-if="error" class="notice danger">❌ {{ error }}</p>
  <ul v-else-if="projects">
    <li v-for="p in projects" :key="p.id">
      <strong>{{ p.title }}</strong>
      <span class="status-badge" :class="p.status === 'done' ? 'is-free' : p.st
      <p>{{ p.description }}</p>
      <small>Équipe : {{ p.team.join(', ') }}</small>
    </li>
  </ul>
</template>
```

3. Directive v-tooltip

`src/directives/tooltip.ts` :

```
import type { Directive, DirectiveBinding } from 'vue';

function updateTooltip(el: HTMLElement, binding: DirectiveBinding<string>)
  el.setAttribute('title', binding.value);
  el.setAttribute('data-tooltip', binding.value);
  el.classList.add('has-tooltip');
}

export const vTooltip: Directive<HTMLElement, string> = {
  mounted: updateTooltip,
  updated: updateTooltip,
  unmounted(el) {
    el.removeAttribute('data-tooltip');
    el.classList.remove('has-tooltip');
  }
};
```

> Le CSS du tooltip (`.has-tooltip`, `.has-tooltip:hover::after`) est déjà inclus dans `src/assets/campus.css` que vous avez copié au TP1 — aucun style à écrire.

4. Enregistrer la directive globalement

`src/main.ts` :

```
import { vTooltip } from './directives/tooltip';

const app = createApp(App);
app.directive('tooltip', vTooltip);
app.use(createPinia()).use(router).mount('#app');
```

5. Utiliser la directive dans RoomCard

```
<template>
  <article class="room-card">
    <h3 v-tooltip="`Capacité : ${room.capacity} places`">{{ room.name }}</h3>
    <button v-tooltip="'Marquer comme favori'" class="fav-btn" @click="onFavori
      {{ room.isFavorite ? '★' : '☆' }}
    </button>
  </article>
</template>
```

6. Type-check global

```
npm run type-check
```

Doit passer sans erreur. Si erreur sur le tooltip global, ajouter dans `src/env.d.ts` :

```
declare module '@vue/runtime-core' {
  interface ComponentCustomProperties {
    vTooltip: typeof import('./directives/tooltip').vTooltip;
  }
}
```

Bonus

- Créer une directive `v-permission="'admin'"` qui cache l'élément si l'utilisateur n'a pas le rôle (lire depuis un store `auth`).
- Écrire un composable `usePagination<T>(items, pageSize)` typé.
- Créer un plugin `installCampusDirectives` qui enregistre toutes les directives custom.

Livrable

- Composable `useCampusApi<T>` typé et réutilisable
- Directive globale `v-tooltip` opérationnelle
- `npm run type-check` sans erreur
- Au moins 3 usages de `v-tooltip` dans l'app

TP suivant : TP9 — Tests Vitest

TP9

Tests Vitest

TP9 — Tests avec Vitest (45 min)

Module associé : M13 — Testing

Objectif

Écrire des tests unitaires pour un composable et un composant. Mocker le store Pinia. Atteindre 50%+ de couverture sur le code métier ajouté.

Pré-requis

- TP8 terminé
- Vitest installé (choix du wizard `create-vue` au TP1)

Étapes

1. Configurer Vitest

Vérifier la présence de `vitest.config.ts` à la racine. Sinon créer :

```
import { fileURLToPath } from 'node:url';
import { mergeConfig, defineConfig, configDefaults } from 'vitest/config';
import viteConfig from './vite.config';

export default mergeConfig(viteConfig, defineConfig({
  test: {
    environment: 'jsdom',
    exclude: [...configDefaults.exclude, 'e2e/**'],
    root: fileURLToPath(new URL('./', import.meta.url)),
    coverage: {
      provider: 'v8',
      reporter: ['text', 'html'],
      include: ['src/**/*.{ts,vue}'],
      exclude: ['src/main.ts', 'src/router/**']
    }
  }
}));
```

Installer le helper Pinia testing :

```
npm install -D @pinia/testing @vue/test-utils
```

2. Tester useSearch (composable)

`src/composables/__tests__/useSearch.test.ts` :

```typescript
import { describe, it, expect } from 'vitest';
import { ref } from 'vue';
import { useSearch } from '../useSearch';

interface Item { name: string; }

describe('useSearch', () => {
  const items = ref<Item[]>([
    { name: 'Amphi Galois' },
    { name: 'Salle 101' },
    { name: 'Amphi Hadamard' }
  ]);

  it('renvoie tous les éléments quand le terme est vide', () => {
    const { filtered } = useSearch(items, 'name');
    expect(filtered.value).toHaveLength(3);
  });

  it('filtre par sous-chaîne, insensible à la casse', () => {
    const { term, filtered } = useSearch(items, 'name');
    term.value = 'amphi';
    expect(filtered.value).toHaveLength(2);

    term.value = 'AMPHI';
    expect(filtered.value).toHaveLength(2);
  });

  it('clear() vide le terme', () => {
    const { term, clear } = useSearch(items, 'name');
    term.value = 'salle';
    clear();
    expect(term.value).toBe('');
  });

  it('renvoie un tableau vide si rien ne correspond', () => {
    const { term, filtered } = useSearch(items, 'name');
    term.value = 'inconnu';
    expect(filtered.value).toEqual([]);
  });
});
```

Lancer :

```
npm run test
```

3. Tester RoomCard (composant)

`src/components/__tests__/RoomCard.test.ts` :

```
import { describe, it, expect } from 'vitest';
import { mount } from '@vue/test-utils';
import RoomCard from '../RoomCard.vue';
import type { Room } from '../../data/rooms';

const baseRoom: Room = {
  id: 1,
  name: 'Amphi Galois',
  building: '510',
  capacity: 80,
  occupancy: 0,
  equipment: ['wifi'],
  isFavorite: false
};

describe('RoomCard.vue', () => {
  it('affiche le nom et le bâtiment', () => {
    const wrapper = mount(RoomCard, { props: { room: baseRoom } });
    expect(wrapper.text()).toContain('Amphi Galois');
    expect(wrapper.text()).toContain('510');
  });

  it('affiche ☆ quand non favori, ★ quand favori', () => {
    const wrapper = mount(RoomCard, { props: { room: baseRoom } });
    expect(wrapper.text()).toContain('☆');

    const fav = mount(RoomCard, { props: { room: { ...baseRoom, isFavorite
    expect(fav.text()).toContain('★');
  });

  it('émet toggle-favorite au clic sur le bouton', async () => {
    const wrapper = mount(RoomCard, { props: { room: baseRoom } });
    await wrapper.get('.fav-btn').trigger('click');
    expect(wrapper.emitted('toggle-favorite')).toEqual([[1]]);
  });

  it('applique la classe is-free quand vide', () => {
    const wrapper = mount(RoomCard, { props: { room: baseRoom } });
    expect(wrapper.classes()).toContain('is-free');
  });

  it('applique is-full quand surchargée', () => {
    const full = { ...baseRoom, occupancy: 75 };
```

```
    const wrapper = mount(RoomCard, { props: { room: full } });
    expect(wrapper.classes()).toContain('is-full');
  });
});
```

4. Tester un composant qui utilise Pinia

`src/views/__tests__/FavoritesView.test.ts` :

```
import { describe, it, expect, vi } from 'vitest';
import { mount } from '@vue/test-utils';
import { createTestingPinia } from '@pinia/testing';
import FavoritesView from '../FavoritesView.vue';
import { useCampusStore } from '../../stores/campus';

describe('FavoritesView.vue', () => {
  it('affiche un message si aucun favori', () => {
    const wrapper = mount(FavoritesView, {
      global: {
        plugins: [createTestingPinia({
          createSpy: vi.fn,
          initialState: { campus: { rooms: [] } }
        })]
      }
    });
    expect(wrapper.text()).toContain('Aucune salle marquée favorite');
  });

  it('affiche les salles favorites', () => {
    const wrapper = mount(FavoritesView, {
      global: {
        plugins: [createTestingPinia({
          createSpy: vi.fn,
          initialState: {
            campus: {
              rooms: [
                { id: 1, name: 'Amphi Galois', isFavorite: true, capacity:
                { id: 2, name: 'Salle 101', isFavorite: false, capacity: 3
              ]
            }
          }
        })]
      }
    });
    expect(wrapper.text()).toContain('Amphi Galois');
    expect(wrapper.text()).not.toContain('Salle 101');
  });
});
```

5. Lancer la couverture

```
npm run test:unit -- --coverage
# Ou si configuré : npm run coverage
```

Ouvrir `coverage/index.html` pour visualiser la couverture par fichier.

Bonus

- Tester `useCampusApi` en mockant `fetch` global avec `vi.stubGlobal`.
- Tester une navigation Vue Router avec `createRouter({ history: createMemoryHistory() })`.
- Ajouter un test E2E Playwright minimaliste : naviguer sur `/rooms`, cliquer sur une carte.

Livrable

- Tests `useSearch.test.ts` passant (4 tests minimum)
- Tests `RoomCard.test.ts` passant (5 tests minimum)
- Tests d'intégration avec Pinia testing
- Couverture des fichiers métier > 50%

🎓 **Fin du dernier TP !**

Place au QCM final, puis au mini-projet d'extension (cf. FINAL_EVALUATION.md).